

|| || ||

•

 $\hat{a} \bullet ' \quad$

•

 $\hat{a} \bullet ' \quad$

•

•

 $\hat{a} \bullet ' \quad$

•

•

•

•

â••‘

 $\hat{a} \bullet ' \quad$ $\hat{a} \bullet ' \quad$ [illegible]

```
scorer.print_scorecard(result)
```

|| || ||

```
from enum import Enum
```

```
__lineage__ = "v12.51 CORNERSTONE + GOD_MODE_V71 + v10.70 + HINT + REGIME"
```

•

[illegible]

```
"base_logit": 1.6877965449124204,
```

```
"adcom_low_penalty": -1e-06,
```


<https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/88909055/eec5aa71-e16a-43bd-b261-4fe9a69b77fe/ULTIMATE ODIN V2-...> 3/18

<https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/88909055/eec5aa71-e16a-43bd-b261-4fe9a69b77fe/ULTIMATE ODIN V2-...> 5/18

<https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/88909055/eec5aa71-e16a-43bd-b261-4fe9a69b77fe/ULTIMATE ODIN V2-...> 6/18

```
class UltimateOdinScorer:
    """
    ULTIMATE ODIN V2.0 â€” THE SPEAR OF ODIN

    Scoring pipeline (7 layers):
    1. v1251 Champion Base (54 features + 6 interactions) â€” IMMUTABLE
```

|| || ||

|| || ||

■ ■ ■

```
module_contributions = {} # Track by module for analysis
```

[illegible]


```

logit = W["base_logit"]
contributions.append(("base_logit", W["base_logit"], "BASE"))

# --- Penalties ---
def _add(cond, key, label="BASE"):
    nonlocal logit
    if cond:
        logit += W[key]
        contributions.append((key, W[key], label))

_add(s.is_snda, "snda_base_penalty")
_add(s.is_snda_pediatric, "snda_pediatric_base_penalty")
_add(s.prior_crl, "prior_crl_penalty")
if s.prior_crl_count > 1:
    extra = (s.prior_crl_count - 1) * W["prior_crl_count_penalty"]
    logit += extra
    contributions.append(("prior_crl_count_penalty", extra, "BASE"))
_add(s.double_crl, "double_crl_penalty")
_add(s.inexperienced_sponsor, "inexperienced_sponsor_penalty")
_add(s.manufacturing_risk, "manufacturing_risk_penalty")
_add(s.form_483, "form_483_penalty")
_add(s.ema_cmc_flag, "ema_cmc_flag_penalty")
_add(s.cmc_extension, "cmc_extension_penalty")
_add(s.adcom_mid, "adcom_mid_penalty")
_add(s.adcom_low, "adcom_low_penalty")
_add(s.pediatric_no_pk, "s22_pediatric_pk_penalty")
_add(s.is_gene_therapy, "gene_therapy_penalty")
_add(s.single_arm, "single_arm_study_penalty")
_add(s.surrogate_endpoint, "surrogate_endpoint_penalty")
if s.safety_severity > 0:
    pen = s.safety_severity * W["safety_severity_penalty"]
    logit += pen
    contributions.append(("safety_severity_penalty", pen, "BASE"))
_add(s.ppm_flag, "ppm_penalty")
_add(s.is_psychedelic, "psychedelics_penalty")
_add(s.is_class1_resubmission, "class1_resubmission_boost")

# Temporal
_add(s.is_hoeg_era, "hoeg_era_constant")
if s.accelerated_approval and s.pdufa_year >= 2025:
    logit += W["accel_approval_2025plus_penalty"]
    contributions.append(("accel_approval_2025plus_penalty",
        W["accel_approval_2025plus_penalty"], "BASE"))
if s.experienced_sponsor and s.pdufa_year >= 2026:
    logit += W["experienced_sponsor_2026_reduction"]
    contributions.append(("experienced_sponsor_2026_reduction",
        W["experienced_sponsor_2026_reduction"], "BASE"))
if s.eu_approved and s.pdufa_year >= 2026:
    logit += W["eu_approved_2026_penalty"]
    contributions.append(("eu_approved_2026_penalty",
        W["eu_approved_2026_penalty"], "BASE"))

# TA penalties
ta_risk = _classify_ta_risk(s.therapeutic_area)
if ta_risk == "HIGH_RISK":
    logit += W["ta_high_risk_penalty"]
    contributions.append(("ta_high_risk_penalty", W["ta_high_risk_penalty"], "BASE"))
elif ta_risk == "MOD_RISK":
    logit += W["ta_mod_risk_penalty"]
    contributions.append(("ta_mod_risk_penalty", W["ta_mod_risk_penalty"], "BASE"))
_add(s.is_pain, "indication_pain_penalty")
_add(s.is_oncology, "indication_onc_boost")
if s.inexperienced_sponsor and ta_risk == "HIGH_RISK":
    logit += W["novice_sponsor_high_risk_ta_penalty"]

```

<https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/88909055/eec5aa71-e16a-43bd-b261-4fe9a69b77fe/ULTIMATE ODIN V...> 10/18

<https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/88909055/eec5aa71-e16a-43bd-b261-4fe9a69b77fe/ULTIMATE ODIN V2...> 11/18

<https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/88909055/eec5aa71-e16a-43bd-b261-4fe9a69b77fe/ULTIMATE ODIN V...> 12/18

<https://ppl-ai-file-upload.s3.amazonaws.com/web/direct-files/attachments/88909055/eec5aa71-e16a-43bd-b261-4fe9a69b77fe/ULTIMATE ODIN V...> 14/18

```

"model": "ULTIMATE_ODIN_V2_SPEAR",
"probability": round(probability, 4),
"v1251_probability": round(v1251_prob, 4),
"delta_from_v1251": round(probability - v1251_prob, 4),
"tier": tier,
"action": tier_info["action"],
"sizing": tier_info["sizing"],
"adjusted_sizing": round(adjusted_sizing, 4),
"exit": tier_info["exit"],
"ta_risk_tier": ta_risk,
"raw_logit": round(raw_logit, 4),
"base_logit_v1251": round(base_logit, 4),
"hint_applied": hint_applied,
"hint_crl_rate": hint_crl_rate,
"market_regime": regime_str,
"regime_multiplier": regime_mult,
"regime_confidence": round(s.regime_confidence, 2),
"positive_signals": positive_signals,
"risk_flags": risk_flags,
"avoid_reasons": avoid_reasons,
"contributions": [(n, round(v, 4), m) for n, v, m in contributions[:20]],
"module_contributions": {k: round(v, 4) for k, v in module_contributions.items()},
"modules_active": {
    "ceo_tone": self.enable_ceo_tone,
    "social_v2": self.enable_social_v2,
    "ops_risk": self.enable_ops_risk,
    "expectation_gap": self.enable_expectation_gap,
    "regime": self.enable_regime,
},
"wf_metrics_base": {"brier": 0.08880, "auc": 0.9082},
}

```

```

def print_scorecard(self, r: Dict, ticker: str = "", drug: str = "",
                    indication: str = "", pdufa_date: str = ""):
    """Pretty-print the ULTIMATE scorecard."""
    W = 78
    print(f"\n{'â••' * W}")
    print(f"  â••i,â•• ULTIMATE ODIN V{self.version} ({self.codename}) â•• PDUFA SCORECARD")
    print(f"{'â••' * W}")
    if ticker:
        print(f"  Ticker: {ticker:12s} Drug: {drug}")
    if indication:
        print(f"  Indication: {indication}")
    if pdufa_date:
        print(f"  PDUFA Date: {pdufa_date}")

    print(f"\n  PROBABILITY: {r['probability']:.1%} (v1251 base: {r['v1251_probability']:.1%},
i">{r['delta_from_v1251']:+.1%})")
    print(f"  Raw Logit: {r['raw_logit']:.3f} (v1251 base logit:
{r['base_logit_v1251']:.3f})")
    if r['hint_applied']:
        print(f"  HINT Blend: Applied (CRL rate: {r['hint_crl_rate']:.1%})")

    tier_emoji = {1: "ðŸŸ¶", 2: "ðŸŸ·", 3: "ðŸŸ¸", 4: "ðŸŸ¸"}
    print(f"\n  â••{ 'â••' * 52 }â••")
    print(f"  â••, {tier_emoji[r['tier']]} TIER {r['tier']:43d}â••,")
    print(f"  â••, ACTION: {r['action']:41s}â••,")
    print(f"  â••, BASE SIZING: {r['sizing']:35s}â••,")
    print(f"  â••, REGIME-ADJ SIZING: {r['adjusted_sizing']:4.2f}x{' ' * 24}â••,")
    print(f"  â••, EXIT: {r['exit']:41s}â••,")
    print(f"  â••, TA RISK: {r['ta_risk_tier']:40s}â••,")
    print(f"  â••, REGIME: {r['market_regime']:>6s} ({r['regime_multiplier']:.1f}x{' ' * 26}â••,")
    print(f"  â••{ 'â••' * 52 }â••")

    if r['positive_signals']:

```

```
def aldx_acid_test():
    """
    ALDX ACID TEST â€” The defining validation case.

    ALDX reproxalap: 2x CRL, CEO bullish on resubmission, FDA quiet review.
    v1251 would output ~TIER_4 (1-5%) due to heavy prior_crl penalties.
    ULTIMATE V2.0 target: TIER_3 (10-15%) due to recovery signals.
    """
    print("\n" + "ðŸ§ª"*35)
    print("  ALDX ACID TEST â€” REPROXALAP RESUBMISSION")
    print("ðŸ§ª"*35)

    scorer = UltimateOdinScorer()

    # ALDX reproxalap signals
    aldx = UltimateSignals(
        # History: 2x CRL (very bearish in v1251)
        prior_crl=True,
        prior_crl_count=2,
        double_crl=True,

        # But recovery signals:
        ceo_tone=CeoTone.BULLISH,      # CEO confident on resubmission
        quiet_review=True,              # FDA not raising new issues
        priority_review=False,
```



```

    fast_track=True,                # Fast track designation

    # Sponsor/TA
    experienced_sponsor=False,      # Mid-size biotech
    inexperienced_sponsor=False,
    therapeutic_area="ophthalmology",
    is_pain=False,

    # Signals
    insider_signal=0.2,              # Mild insider buying
    hiring_signal=0.6,              # NDA-stage hiring detected
    social_signal=0.1,

    # Expanded social
    s17_sentiment=0.3,              # Moderately bullish sentiment
    s18_engagement_spike=0.2,
    s19_social_silence=0.1,         # Not silent (good)
    s20_smart_money_divergence=0.4, # Smart money accumulating

    # Context
    is_hoeg_era=True,
    pdufa_year=2026,
    market_regime=MarketRegime.NORMAL,
)

result = scorer.score(aldx)
scorer.print_scorecard(result, "ALDX", "reproxalap", "Dry Eye Disease", "2026-Q3")

# Acid test assertion
prob = result["probability"]
tier = result["tier"]
print(f"\n {'â€¦'} if tier <= 3 else 'â€¦'} ALDX ACID TEST: Prob={prob:.1%}, Tier={tier}")
print(f" {'â€¦'} if 0.10 <= prob <= 0.25 else 'â€¦'} Target range: 10-25% (TIER_3)")
print(f" v1251 base would give: {result['v1251_probability']:.1%}")
print(f" ULTIMATE V2.0 delta: {result['delta_from_v1251']:+.1%}")

return result

def demo():
    """Run full demo suite."""
    scorer = UltimateOdinScorer()

    print("\n" + "="*78)
    print("  ULTIMATE ODIN V2.0 â€” DEMO SUITE")
    print("="*78)

    # Case 1: Strong approval candidate
    s1 = UltimateSignals(
        btd=True, priority_review=True, experienced_sponsor=True,
        therapeutic_area="oncology", is_oncology=True,
        ceo_tone=CeoTone.BULLISH, quiet_review=True,
        s17_sentiment=0.8, s20_smart_money_divergence=0.6,
        is_hoeg_era=True, pdufa_year=2026,
    )
    r1 = scorer.score(s1)
    scorer.print_scorecard(r1, "ONCO", "oncodrug", "NSCLC 2L", "2026-06-15")

    # Case 2: High risk â€” prior CRL + gene therapy
    s2 = UltimateSignals(
        prior_crl=True, prior_crl_count=1,
        is_gene_therapy=True, manufacturing_risk=True,
        inexperienced_sponsor=True,
        therapeutic_area="rare_disease",
        ceo_tone=CeoTone.CAUTIOUS,

```

```

        amendment_count=2,
        is_hoeg_era=True, pdufa_year=2026,
    )
    r2 = scorer.score(s2)
    scorer.print_scorecard(r2, "GENE", "genetherapy", "Hemophilia B", "2026-09-01")

# Case 3: Gene therapy + experienced sponsor (interaction helps)
s3 = UltimateSignals(
    is_gene_therapy=True, manufacturing_risk=False,
    experienced_sponsor=True, btd=True, orphan=True,
    priority_review=True,
    therapeutic_area="rare_disease",
    ceo_tone=CeoTone.BULLISH,
    is_hoeg_era=True, pdufa_year=2026,
)
r3 = scorer.score(s3)
scorer.print_scorecard(r3, "BIG", "genecure", "SMA Type 1", "2026-07-20")

# Case 4: Bear market regime
s4 = UltimateSignals(
    btd=True, priority_review=True, experienced_sponsor=True,
    therapeutic_area="immunology",
    ceo_tone=CeoTone.NEUTRAL,
    is_hoeg_era=True, pdufa_year=2026,
    market_regime=MarketRegime.BEAR,
)
r4 = scorer.score(s4)
scorer.print_scorecard(r4, "IMMU", "immunodrug", "Atopic Dermatitis", "2026-08-10")

# Case 5: PPM flag (forced TIER 4 â€” ultimate can't save this)
s5 = UltimateSignals(
    ppm_flag=True, priority_review=True, btd=True,
    experienced_sponsor=True,
    therapeutic_area="cns",
    ceo_tone=CeoTone.BULLISH, # Even bullish CEO can't override PPM
    is_hoeg_era=True, pdufa_year=2026,
)
r5 = scorer.score(s5)
scorer.print_scorecard(r5, "FAIL", "failamab", "Alzheimer's", "2026-04-20")

# Summary table
print(f"\n{'â€¢':*78}")
print(f" {'Case':>35s} {'Prob':>7s} {'v1251':>7s} {'Delta':>7s} {'Tier':>5s} {'Action':>15s}")
print(f" {'â€œ':*76}")
cases = [
    ("Strong Onc (BTD+PR+CEO Bull)", r1),
    ("Gene Therapy + Novice + CRL", r2),
    ("Gene Therapy + BigPharma + BTD", r3),
    ("Immunology + Bear Market", r4),
    ("PPM Flag (forced TIER 4)", r5),
]
for label, r in cases:
    print(f" {label:>35s} {r['probability']:>7.1%} {r['v1251_probability']:>7.1%} "
          f"{r['delta_from_v1251']:>7.1%} {r['tier']:>5d} {r['action']:>15s}")
print(f"\n{'â€¢':*78}")

# ALDX Acid Test
aldx_acid_test()

if __name__ == "__main__":
    demo()

```